



UNIVERSIDADE FEDERAL DO CEARÁ – CAMPUS CRATEÚS

Curso: Ciência da Computação

Disciplina: Algoritmos em Grafos

Professor: Rafael Martins Barros

Problema do Caixeiro Viajante

Relatório Técnico

Elislândia Aparecida Horlanda da Silva

Leticia Almeida Lima

Isabelli Araújo Pinho

Crateús – CE, 2026

Sumário

1	Introdução	2
2	Método Proposto	2
2.1	Pré-processamento: matriz de distâncias	2
2.2	Construção das rotas iniciais: Vizinho Mais Próximo com multi-partida	3
2.3	Filtragem de candidatas	3
2.4	Busca local: VND com 2-Opt e Swap	4
2.5	Pseudocódigo	5
2.6	Análise conceitual	6
3	Resultados e Discussões	6
3.1	Instâncias utilizadas	6
3.2	Metodologia de experimentação	7
3.3	Resultados obtidos	7
3.4	Comparação com os valores ótimos conhecidos	8
3.5	Análise dos resultados	9
3.6	Pontos fortes e limitações	10
3.7	Possíveis melhorias	10
4	Conclusão	11
5	Uso de Ferramentas de IA	11

1 Introdução

O Problema do Caixeiro Viajante (PCV) é um dos mais tradicionais e conhecidos problemas de programação matemática [Melamed et al., 1990]. Ele consiste em encontrar um ciclo hamiltoniano de custo mínimo em um grafo ponderado e completo, ou seja, uma rota que visite cada vértice exatamente uma vez e retorne à origem com a menor soma de pesos nas arestas. É um problema indispensável para modelar e resolver problemas complexos de conectividade, otimização de rotas e redes de fluxo. Por ser um problema NP-difícil, a busca por soluções exatas em grafos de grande escala é inviável, demandando o uso de heurísticas e meta-heurísticas [Goldbarg et al., 2017].

Este relatório apresenta o desenvolvimento de um solver para o PCV em linguagem C, projetado como requisito prático para a disciplina de Algoritmos em Grafos da Universidade Federal do Ceará (UFC) – Campus Crateús. O trabalho aborda a implementação e a eficiência de uma abordagem híbrida: a construção de um tour inicial pela Heurística do Vizinho Mais Próximo com estratégia multi-partida [Martí et al., 2013], seguida pelo refinamento via Descida em Vizinhança Variável (VND) [da Cunha et al., 2002], combinando as vizinhanças 2-Opt e Swap [Diadelmo et al., 2024]. Nas seções seguintes, são discutidas as decisões de implementação, os resultados obtidos nos testes com instâncias de benchmark da TSPLIB [Reinelt, 1991] e as análises de desempenho.

2 Método Proposto

O método implementado segue três etapas principais: pré-processamento, construção de rotas iniciais e refinamento por busca local. As subseções a seguir detalham cada uma dessas etapas, junto com as decisões tomadas e os motivos por trás delas.

2.1 Pré-processamento: matriz de distâncias

A primeira coisa que o programa faz é ler as coordenadas das cidades e calcular uma matriz de distâncias. Essa matriz guarda a distância entre cada par de cidades, usando a fórmula EUC_2D:

$$d_{ij} = \left\lceil 0,5 + \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right\rceil \quad (1)$$

Essa fórmula calcula, basicamente, a distância euclidiana entre dois pontos e arredonda para o inteiro mais próximo.

O motivo de calcular tudo de uma vez antes de começar é evitar recalcular raízes quadradas durante a busca local, que acessa distâncias repetidas vezes. Com a matriz pronta, qualquer consulta de distância é feita em tempo constante.

2.2 Construção das rotas iniciais: Vizinho Mais Próximo com multi-partida

A base do método é a heurística do **Vizinho Mais Próximo**. O funcionamento é simples: partindo de uma cidade inicial, a cada passo vai para a cidade mais próxima que ainda não foi visitada. Isso se repete até todas as cidades serem incluídas na rota, e então ela é fechada voltando para a cidade de origem.

O problema dessa heurística é que a qualidade do resultado depende muito de qual cidade é escolhida como ponto de partida, ou seja, uma escolha ruim pode gerar uma rota bem mais cara do que o necessário. Para reduzir essa dependência, o programa executa o Vizinho Mais Próximo partindo de cada uma das n cidades, gerando n rotas candidatas. Essa estratégia é chamada de **multi-partida** [Martí et al., 2013] e ajuda a explorar diferentes possibilidades antes de partir para o refinamento.

2.3 Filtragem de candidatas

Aplicar a busca local em todas as n rotas seria muito demorado para instâncias grandes. Por isso, foi implementado um filtro: para instâncias com $n \leq 80$, todas as rotas iniciais são refinadas; para instâncias com $n > 80$, apenas as **60 rotas de menor custo inicial**, chamadas de “candidatas”, passam para a próxima etapa. As demais são descartadas antes da busca local.

A lógica por trás dessa escolha é que rotas iniciais muito ruins provavelmente continuarão sendo ruins mesmo depois do refinamento, e desperdiçariam tempo que poderia ser melhor aproveitado nas partidas mais promissoras. Os resultados da Seção 3 mostram que essa filtragem reduziu o

tempo de execução de forma expressiva sem afetar a qualidade das soluções nas instâncias testadas.

2.4 Busca local: VND com 2-Opt e Swap

O refinamento das candidatas é feito com o **VND**, uma técnica de busca local que combina vizinhanças como 2-Opt e Swap [da Cunha et al., 2002]:

- **2-Opt:** remove dois arcos da rota e reconecta os segmentos em ordem invertida. A troca é aceita se o peso total diminuir. É a vizinhança mais clássica para o TSP e consegue corrigir cruzamentos na rota.

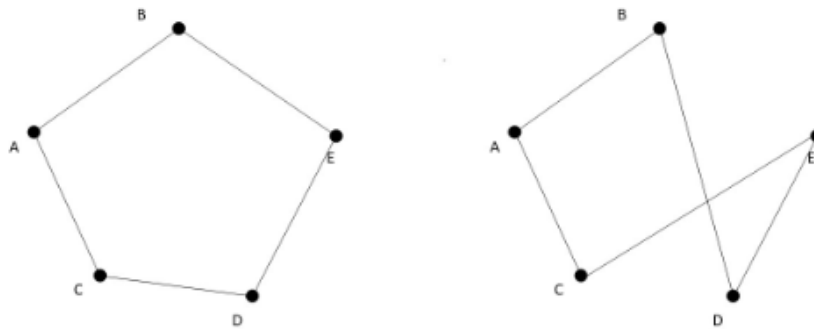


Figura 1: 2-opt. As arestas CD e BE são trocadas pelas arestas BD e CE [Diadelmo et al., 2024]

- **Swap:** troca a posição de duas cidades na rota. É uma operação mais simples que o 2-Opt, mas cobre casos que ele não consegue resolver.

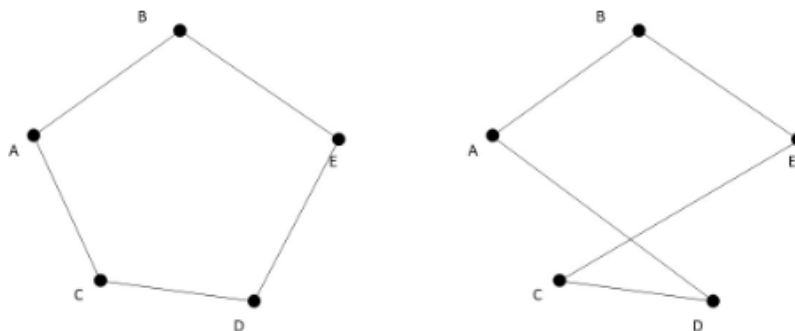


Figura 2: Swap. Os vértices C e D são invertidos na rota e, com isso, as conexões AC e DE tornam-se AD e CE [Diadelmo et al., 2024]

O VND funciona assim: primeiro aplica o 2-Opt repetidamente até não conseguir mais melhorar a solução. Depois tenta o Swap. Se o Swap melhorar, o processo volta para o 2-Opt. Esse ciclo continua até que nenhuma das duas vizinhanças consiga reduzir o peso da rota. Ao final, a melhor rota encontrada entre todas as candidatas é a solução do problema.

2.5 Pseudocódigo

O pseudocódigo a seguir resume o algoritmo completo:

Entrada: instância TSP (formato TSPLIB, tipo EUC_2D)

Saída: rota no formato TOUR

1. ler cidades e dimensao n
2. calcular matriz de distancias D[n][n] pela formula EUC_2D
3. melhorPeso <- infinito (-1), melhorRota <- nulo
4. para cada cidade i de 1 ate n:
 - rota_i <- VizinhoMaisProximo(i, D)
 - peso_i <- calcularPeso(rota_i, D)
 - guardar (rota_i, peso_i) no conjunto de candidatas
5. se n > 80:
 - manter apenas as 60 candidatas de menor peso
6. para cada candidata s no conjunto:
 - repetir:
 - melhorou <- aplicar2Opt(s, D)
 - se nao melhorou:
 - melhorou <- aplicarSwap(s, D)
 - ate que nao melhorou
 - se peso(s) < melhorPeso:
 - melhorPeso <- peso(s)

```
melhorRota <- s
```

7. imprimir melhorRota no formato especificado

2.6 Análise conceitual

A parte mais custosa do algoritmo é a busca local. A construção das rotas pelo Vizinho Mais Próximo tem custo $O(n^2)$ por partida. O 2-Opt e o Swap, por sua vez, verificam $O(n^2)$ pares de trocas a cada iteração.

Sem a filtragem, a busca local era aplicada a todas as n partidas, o que tornava o tempo total muito alto para instâncias grandes. Com o limite de 60 candidatas, o número de execuções fica fixo, e a diferença no tempo de execução é bem visível nos resultados, conforme discutido na seção a seguir.

3 Resultados e Discussões

3.1 Instâncias utilizadas

Os experimentos foram feitos nas seis instâncias disponibilizadas pelo professor, todas no formato TSPLIB [Reinelt, 1991] com coordenadas EUC_2D. A Tabela 1 apresenta as instâncias e seus tamanhos.

Instância	Número de cidades
tsp29	29
tsp38	38
tsp51	51
tsp100	100
tsp150	150
tsp436	436

Tabela 1: Instâncias utilizadas nos experimentos.

O conjunto cobre um intervalo razoável de tamanhos, de 29 a 436 cidades, o que permite observar como o algoritmo se comporta em diferentes escalas.

3.2 Metodologia de experimentação

Os experimentos foram automatizados pelo script `medidor_tempos.py`. O script compila o programa com `gcc -O3`, executa cada instância da pasta `Instancias` e registra o peso total da rota e o tempo de execução. A medição de tempo usa `time.perf_counter` do Python, que tem resolução de nanossegundos.

Os testes foram feitos em uma máquina com Intel Core i5 (10^a geração) e Windows 11. Os tempos registrados podem variar dependendo do hardware e da carga do sistema no momento da medição.

3.3 Resultados obtidos

A Tabela 2 mostra os resultados finais do método com a filtragem de candidatas ativada.

Instância	Peso total	Tempo (s)
tsp29	27603	0,0128
tsp38	6656	0,0108
tsp51	428	0,0173
tsp100	21046	0,1402
tsp150	6636	0,0835
tsp436	1490	1,9611

Tabela 2: Resultados obtidos nas instâncias testadas.

A Tabela 3 compara os tempos de execução antes e depois da adoção da filtragem, mantendo os pesos obtidos como referência.

Instância	Peso antes	Tempo antes (s)	Peso depois	Tempo depois (s)	Redução (%)
tsp29	27603	0,0142	27603	0,0128	9,86
tsp38	6656	0,0125	6656	0,0108	13,60
tsp51	428	0,0358	428	0,0173	51,68
tsp100	21046	0,1874	21046	0,1402	25,19
tsp150	6636	0,3167	6636	0,0835	73,64
tsp436	1490	23,2357	1490	1,9611	91,56

Tabela 3: Impacto da filtragem de candidatas no tempo de execução.

3.4 Comparação com os valores ótimos conhecidos

As instâncias utilizadas correspondem a instâncias clássicas da TSPLIB [Reinelt, 1991]: wi29, dj38, eil51, kroC100, ch150 e pbm436. Os valores ótimos das cinco primeiras foram obtidos pelo professor por meio de métodos exatos (branch-and-bound e branch-and-cut), que garantem matematicamente a otimalidade da solução, porém a um custo computacional muito alto – o dj38, por exemplo, exigiu mais de 3 horas de processamento para apenas 38 cidades. Para pbm436, o valor ótimo conhecido é 1443, conforme indicado pelo próprio enunciado, e o professor utilizou uma heurística para obter 1482.

A Tabela 4 compara apenas a qualidade das soluções, expressa pelo gap percentual em relação ao melhor valor conhecido: $\text{gap} = \frac{C - C^*}{C^*} \times 100$, onde C é o peso obtido pela nossa heurística e C^* é o melhor valor conhecido.

Instância	Nosso peso	Melhor valor conhecido	Gap (%)
tsp29 (wi29)	27603	27603	0,00
tsp38 (dj38)	6656	6656	0,00
tsp51 (eil51)	428	426	+0,47
tsp100 (kroC100)	21046	20749	+1,43
tsp150 (ch150)	6636	6528	+1,65
tsp436 (pbm436)	1490	1443	+3,26

Tabela 4: Comparação entre os resultados obtidos e os melhores valores conhecidos.

O gap ficou abaixo de 1,65% nas cinco instâncias menores. Para as instâncias wi29 e dj38, o peso obtido pela heurística coincidiu com o valor ótimo conhecido, o que não significa que o método garante o ótimo, mas que, nessas instâncias de pequeno porte, a busca local foi capaz de atingi-lo por meio da estratégia multi-partida. Na maior instância (pbm436, 436 cidades), o gap foi de 3,26% em relação ao ótimo publicado, resultado considerado satisfatório para uma abordagem puramente heurística.

3.5 Análise dos resultados

O algoritmo gerou rotas válidas para todas as instâncias. Para as menores (até 51 cidades), o tempo ficou abaixo de 0,02 segundos, quase instantâneo. Mesmo a maior instância, tsp436, foi resolvida em menos de 2 segundos com a filtragem ativa e a avaliação local de custo (Delta).

O resultado mais importante foi o impacto combinado da filtragem de candidatas com o cálculo de Delta em $O(1)$. Na instância tsp436, o tempo caiu de 23,24 segundos para 1,96 segundos, uma redução de quase 92%. Inclusive, nos primeiros testes, o tsp436 chegou a alcançar 124,31 segundos antes das otimizações. Enquanto a filtragem reduziu a quantidade de rotas submetidas ao VND, o cálculo do Delta evitou o recálculo completo da rota $O(n)$ a cada tentativa do Swap e do 2-Opt, limitando as somas apenas às arestas afetadas pela troca. O ponto mais relevante é que o peso da solução ficou exatamente o mesmo (1490) nos dois casos, mostrando que filtrar as

candidatas ruins não prejudicou a qualidade do resultado.

As reduções de tempo nas instâncias menores foram mais modestas, o que também faz sentido: quando n é pequeno, o custo total da busca já é baixo, e essas otimizações conjuntas têm menos espaço para fazer diferença perceptível em escala de segundos.

3.6 Pontos fortes e limitações

Como pontos fortes, o método é simples de entender e implementar, não exige ajuste de parâmetros complexos e funciona para qualquer instância TSPLIB do tipo EUC_2D. A filtragem de candidatas foi uma melhoria prática que funcionou bem sem aumentar a complexidade do código.

Como limitação, o método é determinístico, sempre produz o mesmo resultado para a mesma instância, e pode ficar preso em ótimos locais sem conseguir escapar. A filtragem também pode, em teoria, descartar uma candidata que teria levado a uma solução melhor depois do VND. Isso não aconteceu nas instâncias testadas, mas pode ocorrer em instâncias com geometrias mais complexas.

3.7 Possíveis melhorias

Uma melhoria de tempo seria usar **listas de candidatos** no 2-Opt [Bentley, 1992]: ao invés de testar todos os $O(n^2)$ pares, testa-se apenas os k vizinhos mais próximos de cada cidade (por exemplo, $k = 10$ ou $k = 20$). Isso reduz bastante o custo de cada iteração, mas exige manter listas ordenadas de vizinhos, o que complicaria um pouco o código.

Uma melhoria de qualidade seria aplicar uma perturbação chamada **double-bridge** e rodar a busca local novamente. Essa operação desfaz parte da rota de um jeito que o 2-Opt sozinho não consegue desfazer, ajudando a escapar de ótimos locais. Isso transformaria o método em uma Busca Local Iterada (ILS), que em geral produz resultados bem melhores [Alves et al.,]. O custo é mais código, mais aleatoriedade e parâmetros adicionais para ajustar, o que exigiria mais conhecimento técnico que não possuímos.

4 Conclusão

A execução deste trabalho permitiu aplicar, de forma prática, conceitos de otimização combinatória e análise de algoritmos estudados na disciplina de Algoritmos em Grafos. O solver desenvolvido em linguagem C demonstrou que a escolha adequada das estruturas de dados e dos operadores de busca é determinante para resolver problemas NP-difíceis de forma eficiente.

O método implementado combinou a heurística do Vizinho Mais Próximo com estratégia multi-partida, filtragem das melhores candidatas e refinamento por VND com as vizinhanças 2-Opt e Swap. A comparação com os valores ótimos conhecidos das instâncias da TSPLIB mostrou que a abordagem produz soluções de boa qualidade: o gap ficou abaixo de 1,65% nas cinco instâncias menores, e em 3,26% na maior instância (pbm436, com 436 cidades), resultado obtido em menos de 2 segundos. Nas instâncias wi29 e dj38, a heurística atingiu o mesmo valor do ótimo conhecido, o que demonstra a eficácia da estratégia multi-partida em instâncias de pequeno porte.

A otimização de maior impacto foi a filtragem de candidatas: ao restringir o VND às 60 rotas iniciais de menor custo, o tempo de execução do tsp436 caiu de 23,24 segundos para 1,96 segundos, redução de 91,56%, sem qualquer perda de qualidade nas instâncias testadas.

Conclui-se que o projeto evidenciou a importância de equilibrar qualidade de solução e eficiência computacional. Para trabalhos futuros, a incorporação de listas de candidatos no 2-Opt [Bentley, 1992] e de perturbações como o double-bridge em uma Busca Local Iterada representariam os próximos passos naturais para reduzir ainda mais o gap em instâncias maiores.

5 Uso de Ferramentas de IA

Em conformidade com a Seção 8 do documento de especificações do trabalho (Portaria N° 39/PRPPG/UFC), a equipe declara o uso de ferramentas de Inteligência Artificial Generativa como apoio pedagógico e de otimização, sem delegação da compreensão crítica.

- **Ferramentas Utilizadas:** ChatGPT/Codex e Google Gemini.

- **Finalidades:**

1. Identificação de gargalos de desempenho e sugestão do cálculo de *Delta* em $O(1)$ para

as vizinhanças.

2. Estruturação da lógica da matriz de distâncias.
3. Auxílio na sugestão de criação do script em Python (`medidor_tempos.py`) para automação da leitura em lote.
4. Auxílio na formatação (já que utilizamos o **Overleaf**) e organização semântica das seções deste relatório.

• **Prompts relevantes utilizados:**

- “*Como posso otimizar a minha função de Swap no TSP em C para não ter que recalcular a rota inteira?*”
- “*Existe alguma forma de não rodar o 2-opt em todas as cidades iniciais sem perder muita qualidade? (Geração da ideia do filtro de candidatas).*”

Os dados experimentais apresentados na Seção 3 foram gerados localmente pelo script na máquina da nossa equipe, garantindo a veracidade computacional do estudo.

Referências

- [Alvesa et al.,] Alvesa, D. J. F., Avelino, B., de Faria, V. C. N., Macedo, E. A., Martini, V. G., Vieira, B. S., and Chaves, A. A. Estudo comparativo de metaheurísticas aplicadas ao problema do caixeiro viajante multiplo.
- [Bentley, 1992] Bentley, J. J. (1992). Fast algorithms for geometric traveling salesman problems. *ORSA Journal on computing*, 4(4):387–411.
- [da Cunha et al., 2002] da Cunha, C. B., de Oliveira Bonasser, U., and Abrahão, F. T. M. (2002). Experimentos computacionais com heurísticas de melhorias para o problema do caixeiro viajante. In *XVI Congresso da Anpet*.
- [Diadelmo et al., 2024] Diadelmo, M. V., Batista, L. S., and Bessani, M. (2024). Uma análise estatística de estruturas de vizinhança para o problema do caixeiro viajante. In *Congresso Brasileiro de Automática-CBA*, volume 4.

- [Goldbarg et al., 2017] Goldbarg, E., Goldbarg, M., and Luna, H. (2017). *Otimização combinatória e metaheurísticas: algoritmos e aplicações*. Elsevier Brasil.
- [Martí et al., 2013] Martí, R., Resende, M. G., and Ribeiro, C. C. (2013). Multi-start methods for combinatorial optimization. *European Journal of Operational Research*, 226(1):1–8.
- [Reinelt, 1991] Reinelt, G. (1991). Tsplib—a traveling salesman problem library. *ORSA journal on computing*, 3(4):376–384.